



資料分析與機器學習 (Data Analysis and Machine Learning)

在所有可行的方案中，找出對特定資料集
泛化能力最好、預測誤差最小的模型

Lecture 6 –

Model Selection and Evaluation & Scikit-learn

教師：陳慶永

Outline

- Model Selection
 - ▶ Cross-Validation (交叉驗證)
 - ▶ Hyperparameter Tuning
- Model Evaluation
 - ▶ Confusion Matrix (混淆矩陣)
 - ▶ Performance Metrics
- Scikit-learn
 - ▶ model_selection
 - ▶ metrics
- Tuning and Evaluating a Classifier with Scikit-learn
 - ▶ Cross-Validation based Grid Search (格狀搜尋)
- Classwork
 - ▶ Assignment #3



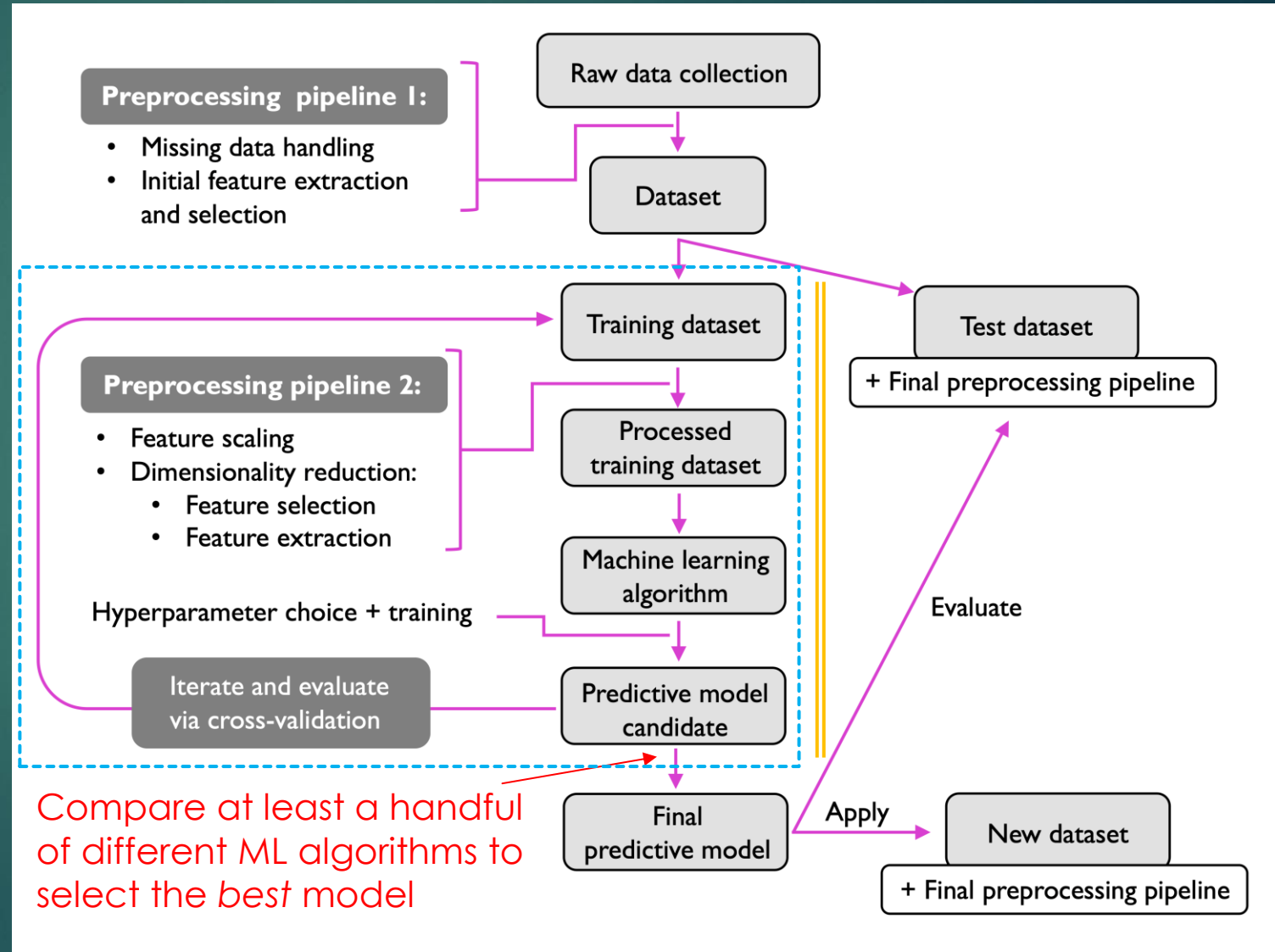
Model Selection

Model Selection

- **Hyperparameter Tuning** (in the narrow sense)
 - ▶ Select the optimal values of tuning parameters (hyperparameters) for a specified model/algorithm
 - ▶ Tuning and comparing different parameter settings to further improve the performance for making predictions on unseen data
- **Algorithm Selection** (in the broad sense)
 - ▶ Select the best model/algorithm
 - ▶ Comparing the optimal performance of different models/algorithms

How to Validate a Model

- Test dataset (✗)
- Training dataset
 - ▶ KNN classifier
 - Instance-based
 - $k = 1$



Cross-Validation

- Techniques to obtain *reliable estimates* of the model's generalization performance
 - ▶ Holdout cross-validation
 - ▶ k-fold cross-validation

Holdout Cross-Validation

- Separate the data into three parts:

- ▶ Training dataset

- Model fitting

- ▶ Validation dataset

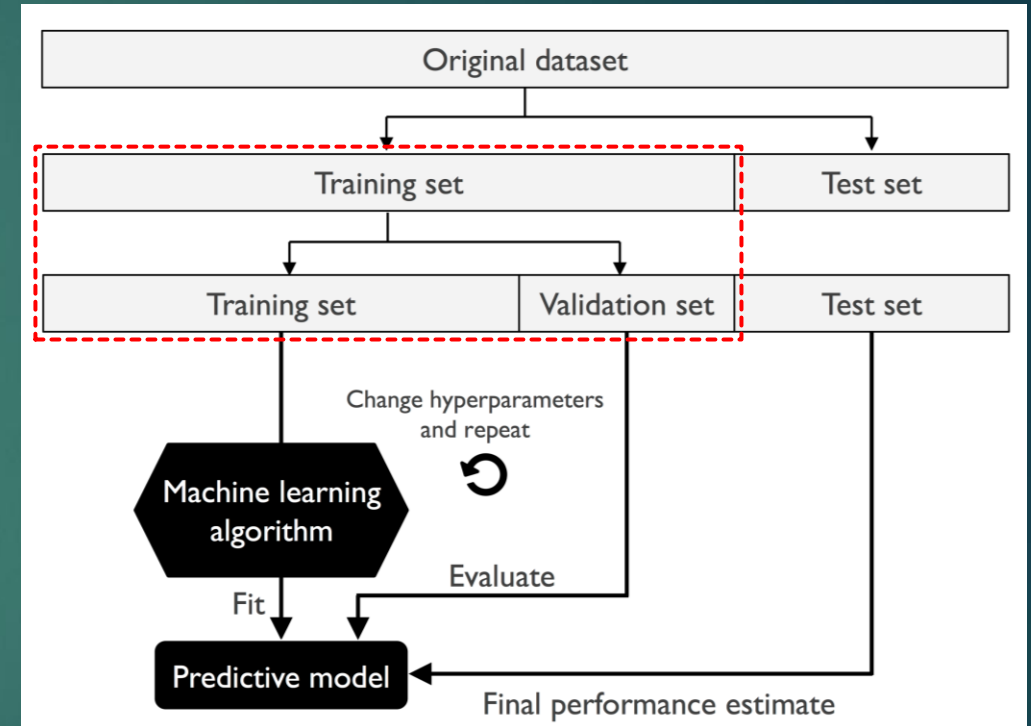
- Model evaluation
 - Model selection

- ▶ Test dataset

- Model evaluation
 - Generalization performance estimate

- Disadvantage

- ▶ Sensitive to how we partition the training dataset into the training and validation subsets



k-fold Cross-Validation

- Randomly split the training dataset into k folds without replacement
 - ▶ $k-1$ training folds used for the model training
 - ▶ 1 test fold used for performance evaluation
- This procedure is repeated k times so that we obtain k models and performance estimates
 - ▶ Average performance of k models is less sensitive to the sub-partitioning of training data

k-fold Cross-Validation

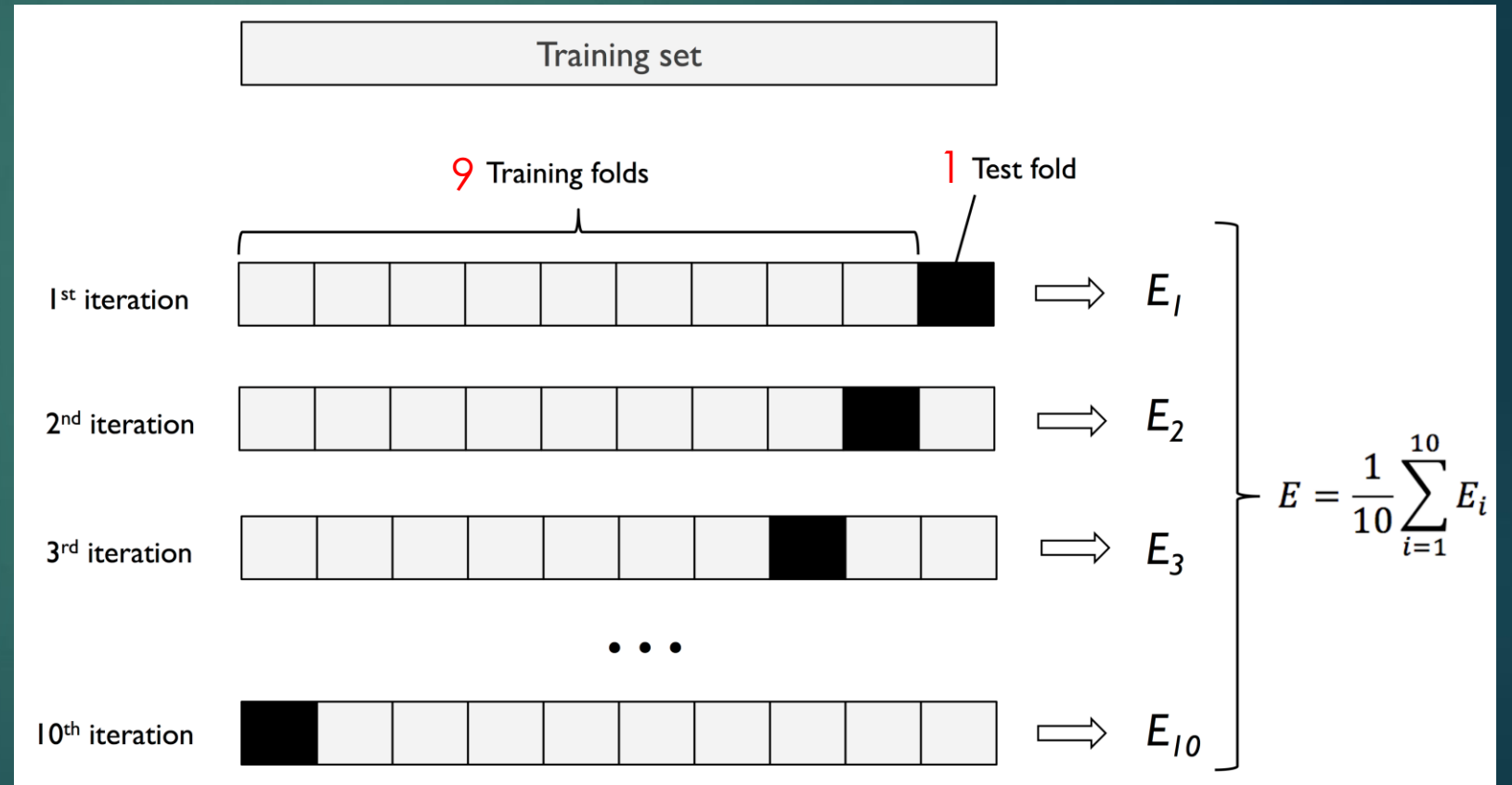
■ Example: $k = 10$ (a good standard value)

▶ Small dataset

- Large k

▶ Large dataset

- Small k



A special case is the *leave-one-out* cross-validation method (k equals to number of training samples). 9

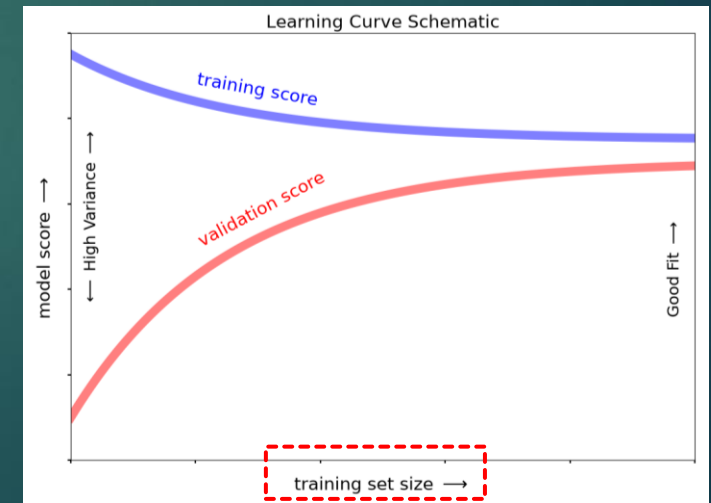
k-fold Cross-Validation

- Once found satisfactory hyperparameter values, retrain the model on the *complete* training dataset
 - ▶ Typically interested in a single, final model (versus k individual models)
 - ▶ More training examples usually results in a more accurate and robust model
- *Stratified* k-fold cross-validation
 - ▶ Class label proportions are preserved in each fold

Learning Curves (學習曲線)

- Plot of the model training and validation scores (e.g., accuracies) v.s. **training dataset size**
 - ▶ Whether the model suffers from high variance (overfitting) or high bias (underfitting)
 - ▶ Whether the collection of more data could help to address this problem

While collecting more training data *usually* tends to decrease the chance of overfitting, it *may not* always help, for example, if the training data is extremely noisy or the model is already very close to optimal.



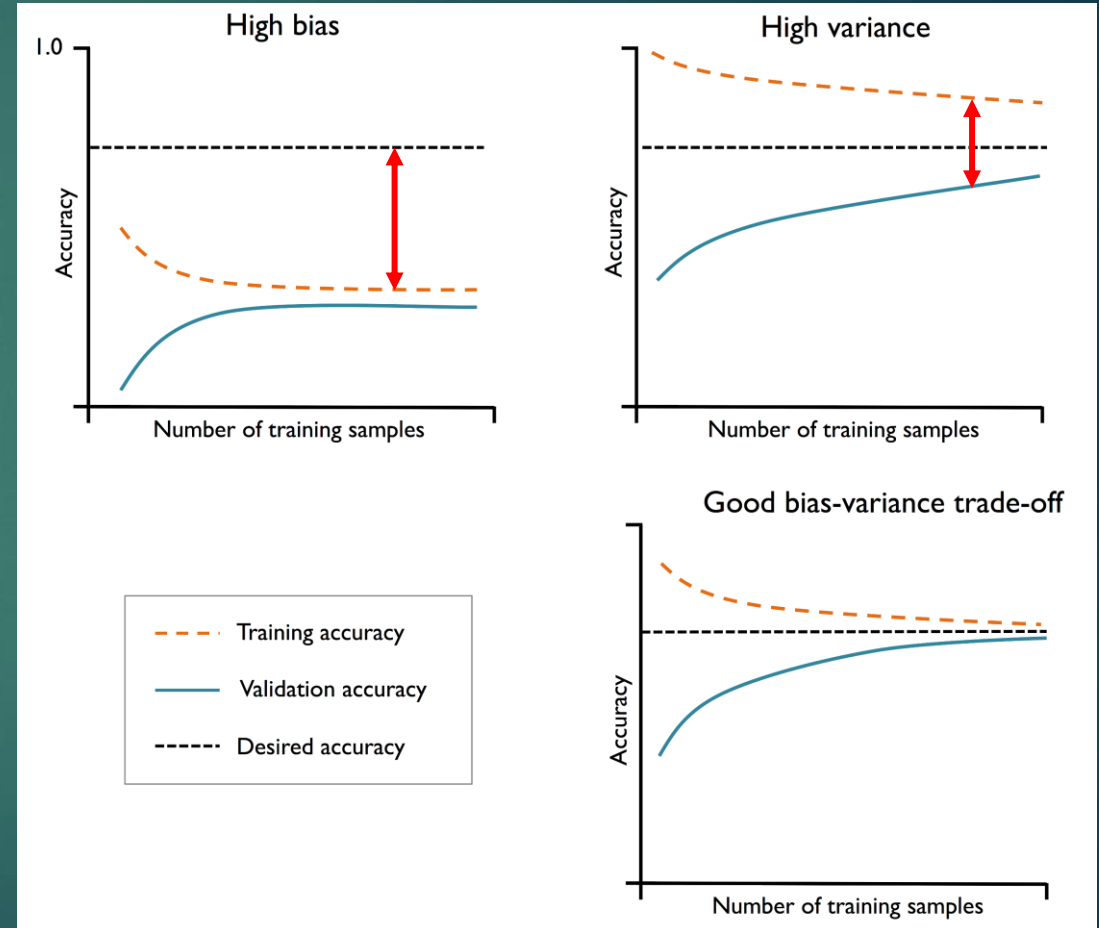
Learning Curves (學習曲線)

■ High bias

- ▶ Both **low** training and cross-validation accuracy
- ▶ Solutions:
 - Increase the number of model parameters
 - Decrease the degree of regularization

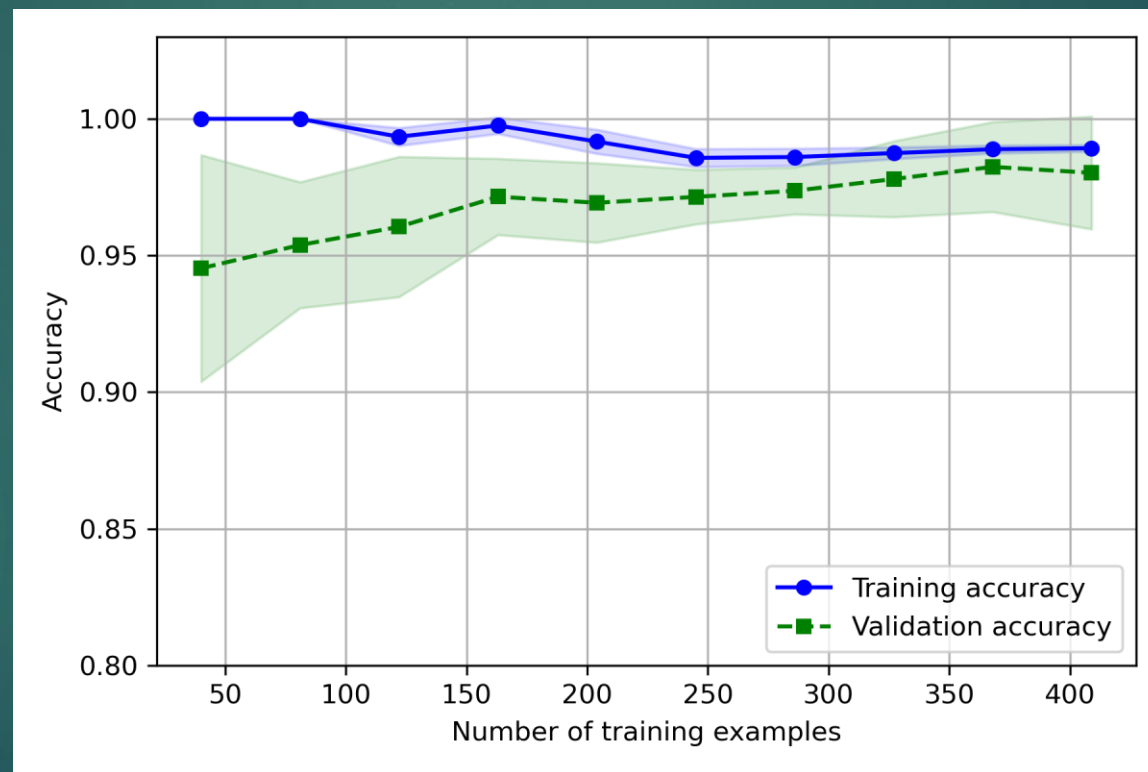
■ High variance

- ▶ Large **gap** between the training and cross-validation accuracy
- ▶ Solutions:
 - Collect more training data
 - Reduce the complexity of the model
 - Increase the regularization parameter
 - Decrease the number of features



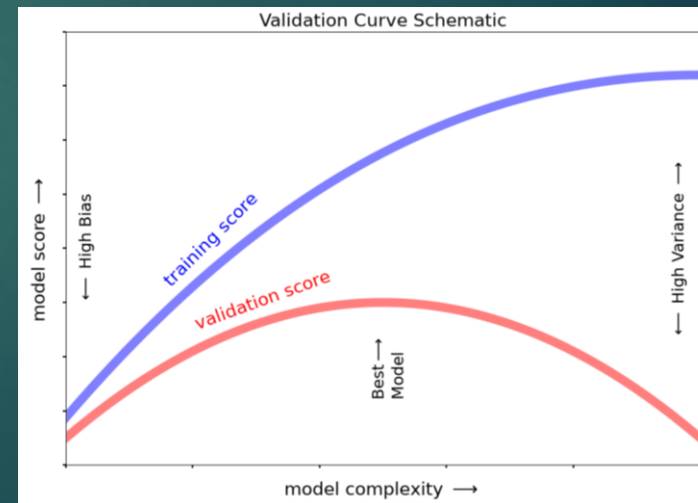
Learning Curves (學習曲線)

■ Example



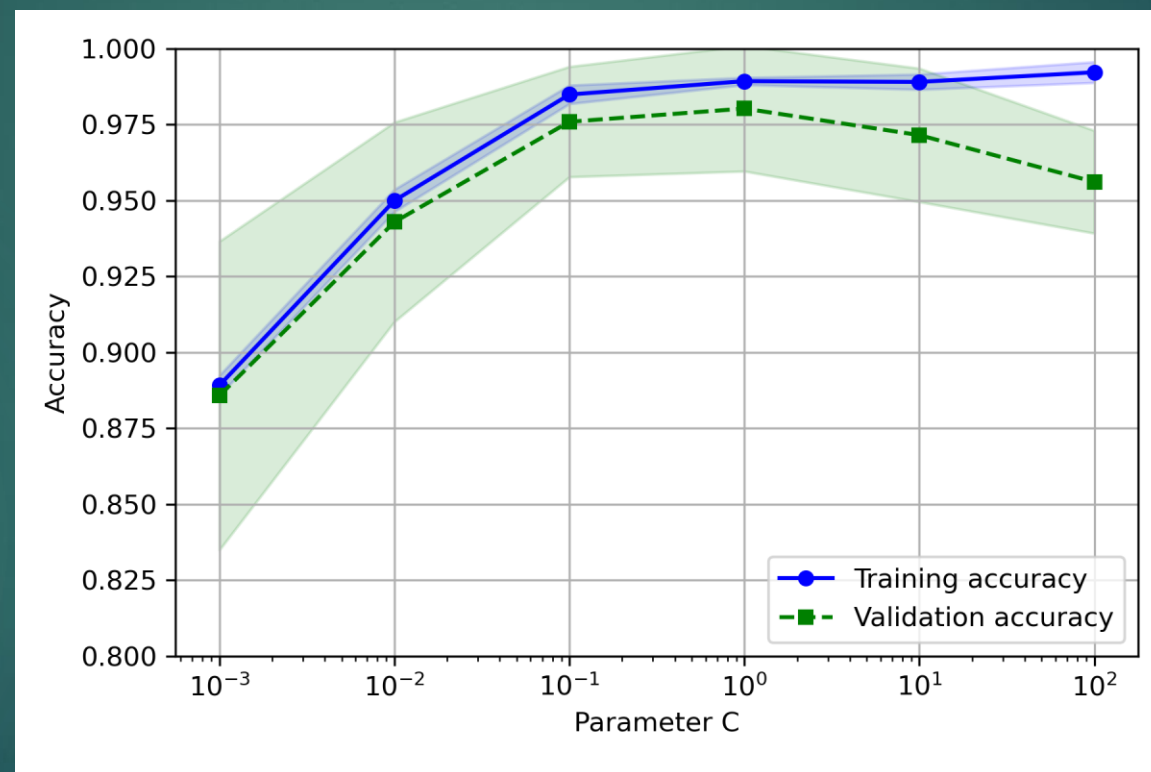
Validation Curves (驗證曲線)

- Plot of the model training and validation scores (e.g., accuracies) v.s. **values of hyperparameters**
 - ▶ Whether the model suffers from high variance (overfitting) or high bias (underfitting)
 - ▶ Choosing optimal values of hyperparameters (e.g., regularization parameter)



Validation Curves (驗證曲線)

■ Example



Hyperparameter Tuning

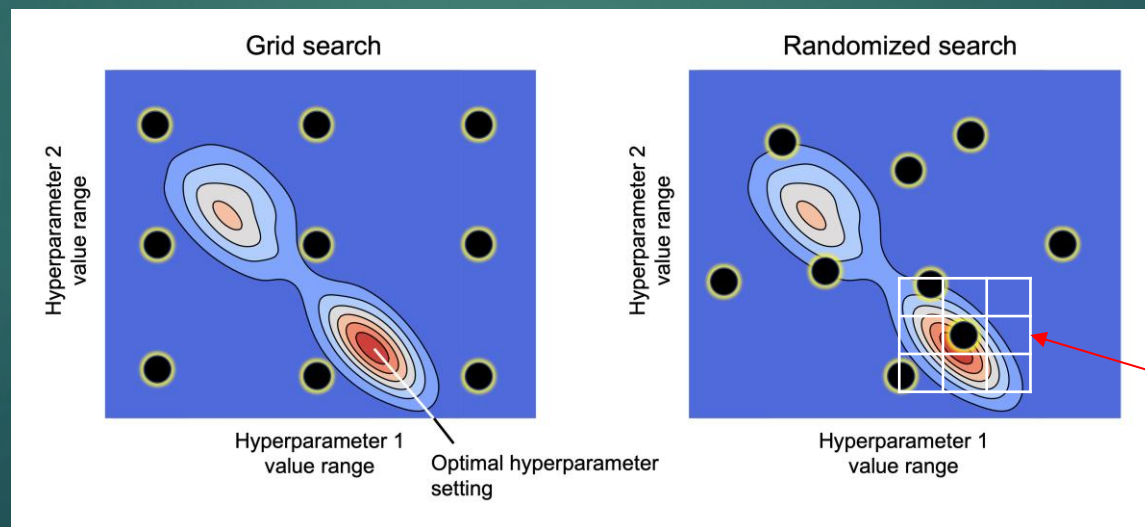
- Validation curves can be used to improve the performance of a model by tuning **one** of its hyperparameters
- How to find the (sub-)**optimal combination** of hyperparameter values?
 - ▶ Cross-validation + **search strategy**
 - Grid search
 - Randomized search
 - Successive halving search

Grid Search

- A *brute-force exhaustive search* paradigm
 - ▶ Specify a list of values for different hyperparameters
 - Forming a set of all combinations (called **grid**) of different values of hyperparameters
 - ▶ Evaluates the model performance (via cross-validation) for *each* combination to obtain the optimal combination of values from this grid
- Guaranteed to find the *optimal* hyperparameter configuration *if* it is contained in the specified grid
 - ▶ Specifying large hyperparameter grids makes grid search *very expensive* in practice

Randomized Search

- *Sampling* different parameter combinations
 - ▶ Draw hyperparameter configurations *randomly* from distributions (or discrete sets)
 - ▶ Explore a *wider range* of hyperparameter value settings in a more cost- and time-effective manner



Successive Halving Search

- More resource-efficient hyperparameter search

- ▶ The following steps are repeated until only one hyperparameter configuration remains

1. Draw a large set of candidate configurations via random sampling
2. Train the models with limited resources, for example, a small subset of the training data (as opposed to using the entire training set)
3. Discard the bottom 50 percent based on predictive performance
4. Go back to *step 2* with an increased amount of available resources

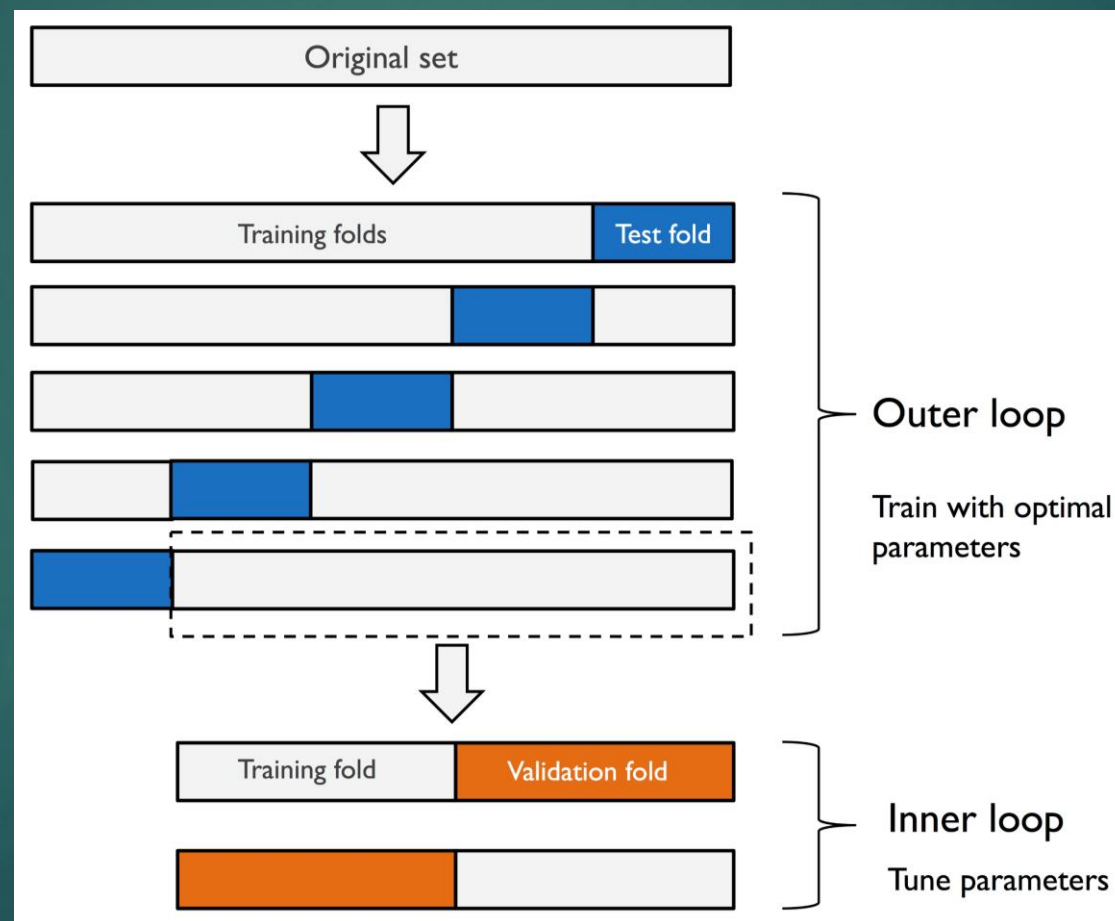
Algorithm Selection

■ Nested Cross-Validation (巢狀交叉驗證)

- ▶ Outer k-fold cross-validation loop to split the data into training and test folds
 - Inner loop is used to select the model using k-fold cross-validation on the training fold
 - After model selection, the test fold is then used to evaluate the model performance
- ▶ A good estimate of what to expect if we tune the hyperparameters of a model and use it on unseen data

Algorithm Selection

■ Example: 5 × 2 cross-validation



Average performance
used for algorithm
selection

Model selection



Model Evaluation

Confusion Matrix

- A square matrix that reports the **counts** of predictions of a classifier
 - ▶ True Positive (TP)
 - ▶ True Negative (TN)
 - ▶ False Positive (FP)
 - ▶ False Negative (FN)

		Predicted class	
		P	N
Actual class	P	True positives (TP)	False negatives (FN)
	N	False positives (FP)	True negatives (TN)

Performance Metrics

- Prediction Error (ERR) (錯誤率)

$$ERR = \frac{FP + FN}{FP + FN + TP + TN}$$



- Accuracy (ACC) (正確率)

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} = 1 - ERR$$



Performance Metrics

- True Positive Rate (TPR) (真陽率)



$$TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$



- False Positive Rate (FPR) (偽陽率) Useful for imbalanced class problems

$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$



Performance Metrics

■ Precision (PRE) (精確率)

$$PRE = \frac{TP}{TP + FP}$$

■ Recall (REC) (召回率)

$$REC = TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

Performance Metrics

■ F1 Score

▶ Harmonic mean of PRE and REC

$$F1 = 2 \frac{PRE \times REC}{PRE + REC}$$

■ Matthews Correlation Coefficient (MCC) (馬修斯相關係數 或 Phi 相關係數)

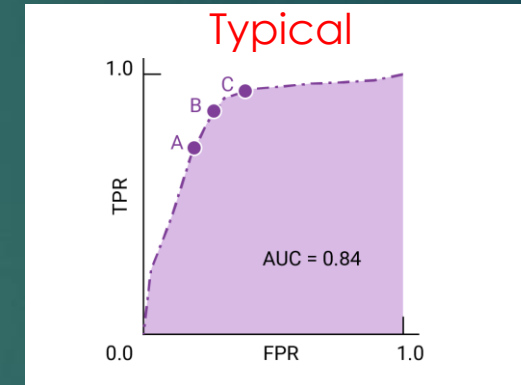
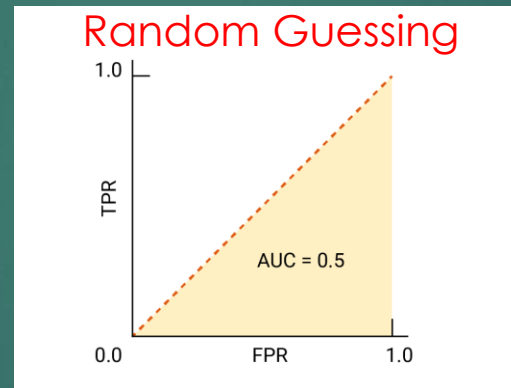
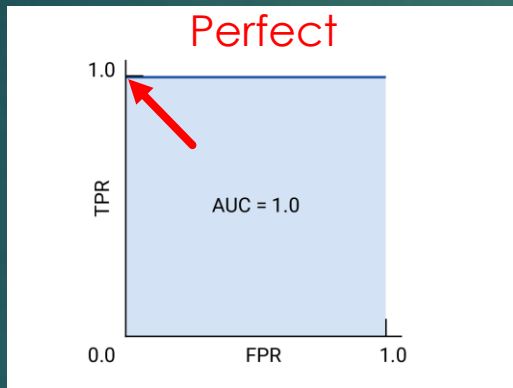
$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

$$-1 \leq MCC \leq 1$$

Performance Metrics

■ Receiver Operating Characteristic (ROC)

- ▶ Curve of **FPR v.s. TPR**, which are computed by *shifting the decision threshold* of the classifier



■ ROC Area Under the Curve (ROC AUC) ↑

- ▶ **Area** under the ROC curve $[0,1]$

Metrics for Multiclass Classification

k-class ($k > 2$)

■ Micro-average (微平均)

▶ Weight each *instance* (or prediction) equally

▶ Example:

$$PRE_{micro} = \frac{TP_1 + \dots + TP_k}{TP_1 + \dots + TP_k + FP_1 + \dots + FP_k}$$

關注「每一個樣本」
(由“多數樣本”類別
性能主導)

■ Macro-average (宏平均) (Useful for class imbalances)

▶ Weight all *classes* equally

▶ Example:

$$PRE_{macro} = \frac{PRE_1 + \dots + PRE_k}{k}$$

關注「每一個類別」
(所有類別一視同仁)

Class Imbalance (類別不平衡)

- One class or multiple classes (called majority class) are *over-represented* in a dataset
 - ▶ When we fit classifiers on such datasets
 - Focus on other *metrics* (e.g., precision or recall) than accuracy
 - Upsample the minority class, downsample the majority class, and generate *synthetic* training examples
 - Assign a larger penalty to wrong predictions on the minority class (setting `class_weight='balanced'` in scikit-learn classifiers)

當模型猜錯一個「小類」的樣本時，它所受到的懲罰 (Loss) 會乘以該權重，使其影響力等同於多個大類樣本



Scikit-learn

General Modules

- datasets
 - preprocessing, impute, compose
 - pipeline, feature_selection, decomposition
 - model_selection
 - metrics
- } 模型選擇與評估

Scikit-learn functions and classes often have parameters with default values that we *may* omit for clarity. You can read more about those parameters by going through the online documentation.

sklearn.model_selection

- Tools for model selection, such as cross validation and hyper-parameter tuning

► Example: **StratifiedKFold** class

- Class-wise stratified K-Fold cross-validator
- Provides *train/test indices* to split data in train/test sets

```
>>> import numpy as np
>>> from sklearn.model_selection import StratifiedKFold
➡ >>> kfold = StratifiedKFold(n_splits=10).split(X_train, y_train)
>>> scores = []
>>> for k, (train, test) in enumerate(kfold):
...     pipe_lr.fit(X_train[train], y_train[train]) Training
...     score = pipe_lr.score(X_train[test], y_train[test]) Validation
...     scores.append(score)
```

shuffle (bool, default=False): Whether to shuffle each class's samples before splitting into batches.

sklearn.model_selection

► Example: `cross_val_score()` function

- Evaluate a score using stratified k-fold cross-validation

```
>>> from sklearn.model_selection import cross_val_score
➔ >>> scores = cross_val_score(estimator=pipe_lr,
...                             X=X_train,
...                             y=y_train,
...                             cv=10, ← k
...                             n_jobs=1) ← Number of CPUs
>>> print(f'CV accuracy scores: {scores}')
```

CV accuracy scores: [0.93478261 0.93478261 0.95652174
0.95652174 0.93478261 0.95555556
0.97777778 0.93333333 0.95555556
0.95555556]

scoring (str or callable, default=None): Scoring method to use to evaluate.

Setting `n_jobs = -1` uses *all* available CPUs on our machine to do the computation in parallel.

sklearn.model_selection

► Example: `learning_curve()` function

- Learning curve that determines cross-validated training and test scores for different training set sizes

```
>>> import matplotlib.pyplot as plt
>>> from sklearn.model_selection import learning_curve
>>> pipe_lr = make_pipeline(StandardScaler(),
...                          LogisticRegression(penalty='l2',
...                                              max_iter=10000))
→ >>> train_sizes, train_scores, test_scores = \
...     learning_curve(estimator=pipe_lr,
...                     X=X_train,
...                     y=y_train,
...                     train_sizes=np.linspace(
...                         0.1, 1.0, 10),
...                     cv=10,
...                     n_jobs=1)
>>> train_mean = np.mean(train_scores, axis=1)
>>> train_std = np.std(train_scores, axis=1)
>>> test_mean = np.mean(test_scores, axis=1)
>>> test_std = np.std(test_scores, axis=1)
```

Training dataset sizes

sklearn.model_selection

► Example: `validation_curve()` function

- Validation curve that determines training and test scores for varying parameter values

```
>>> from sklearn.model_selection import validation_curve
>>> param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
>>> train_scores, test_scores = validation_curve(
...     estimator=pipe_lr,
...     X=X_train,
...     y=y_train,
...     param_name='logisticregression__C',
...     param_range=param_range,
...     cv=10)
>>> train_mean = np.mean(train_scores, axis=1)
>>> train_std = np.std(train_scores, axis=1)
>>> test_mean = np.mean(test_scores, axis=1)
>>> test_std = np.std(test_scores, axis=1)
```

Varying parameter values

C: The inverse regularization parameter

Inside a Pipeline, an estimator's `parameter` can be specified using `'__'` syntax.

sklearn.model_selection

► Example: **GridSearchCV** class

- Exhaustive search over specified parameter values for an estimator

```
>>> from sklearn.model_selection import GridSearchCV
>>> from sklearn.svm import SVC
>>> pipe_svc = make_pipeline(StandardScaler(),
...                           SVC(random_state=1))
>>> param_range = [0.0001, 0.001, 0.01, 0.1,
...                 1.0, 10.0, 100.0, 1000.0]
>>> param_grid = [{'svc__C': param_range,
...                 'svc__kernel': ['linear']},
...               {'svc__C': param_range,
...                 'svc__gamma': param_range,
...                 'svc__kernel': ['rbf']}]
>>> gs = GridSearchCV(estimator=pipe_svc,
...                    param_grid=param_grid,
...                    scoring='accuracy',
...                    cv=10,
...                    refit=True,
...                    n_jobs=-1)
>>> gs = gs.fit(X_train, y_train)
>>> print(gs.best_score_)
0.9846153846153847
>>> print(gs.best_params_)
{'svc__C': 100.0, 'svc__gamma': 0.001, 'svc__kernel': 'rbf'}
```

dict or list of dictionaries

Refit an estimator (gs.best_estimator_) using the best found parameters on the whole dataset

```
>>> clf = gs.best_estimator_
>>> clf.fit(X_train, y_train) not necessary
>>> print(f'Test accuracy: {clf.score(X_test, y_test):.3f}')
Test accuracy: 0.974
```

sklearn.model_selection

► Example: **RandomizedSearchCV** class

- Randomized search on hyper parameters

- A fixed number of parameter settings is sampled from the specified distributions
- **Distributions** must provide a **rvs** method for sampling (such as those from [scipy.stats.distributions](#))

➡ `>>> param_range = scipy.stats.loguniform(0.0001, 1000.0)`

```
>>> np.random.seed(1)
```

```
>>> param_range.rvs(10)
```

```
array([8.30145146e-02, 1.10222804e+01, 1.00184520e-04, 1.30715777e-02,  
       1.06485687e-03, 4.42965766e-04, 2.01289666e-03, 2.62376594e-02,  
       5.98924832e-02, 5.91176467e-01])
```

sklearn.model_selection

```
>>> from sklearn.model_selection import RandomizedSearchCV
>>> pipe_svc = make_pipeline(StandardScaler(),
...                           SVC(random_state=1))
>>> param_grid = [{'svc__C': param_range,
...                 'svc__kernel': ['linear']},
...                {'svc__C': param_range,
...                 'svc__gamma': param_range,
...                 'svc__kernel': ['rbf']}]
>>> rs = RandomizedSearchCV(estimator=pipe_svc,
...                           param_distributions=param_grid,
...                           scoring='accuracy',
...                           refit=True,
...                           n_iter=20,
...                           cv=10,
...                           random_state=1,
...                           n_jobs=-1)

>>> rs = rs.fit(X_train, y_train)
>>> print(rs.best_score_)
0.9670531400966184

>>> print(rs.best_params_)
{'svc__C': 0.05971247755848464, 'svc__kernel': 'linear'}
```

dict or list of dictionaries

Similar to GridSearchCV, except that we use distributions for specifying parameter ranges

Number of parameter settings that are sampled

sklearn.model_selection

► Example: **HalvingRandomSearchCV** class

- Randomized search on hyper parameters.
- Starts evaluating *all* the candidates with a small amount of resources and iteratively selects the best candidates, using more and more resources (白話：練蠱)

```
>>> from sklearn.model_selection import HalvingRandomSearchCV

>>> hs = HalvingRandomSearchCV(pipe_svc,
...                             param_distributions=param_grid,
...                             n_candidates='exhaust',
...                             resource='n_samples',
...                             factor=1.5,
...                             random_state=1,
...                             n_jobs=-1)
```

The number of candidate parameters to sample, at the first iteration

'Halving' parameter determines the proportion of candidates that are selected for each subsequent iteration

sklearn.model_selection

■ Nested cross-validation with grid search

▶ `cross_val_score(GridSearchCV)`

```
>>> param_range = [0.0001, 0.001, 0.01, 0.1,
...                 1.0, 10.0, 100.0, 1000.0]
>>> param_grid = [{'svc__C': param_range,
...                 'svc__kernel': ['linear']},
...                {'svc__C': param_range,
...                 'svc__gamma': param_range,
...                 'svc__kernel': ['rbf']}]
→ >>> gs = GridSearchCV(estimator=pipe_svc,
...                       Inner k-fold + grid search param_grid=param_grid,
...                       scoring='accuracy',
...                       cv=2)
```

Outer k-fold

```
→ >>> scores = cross_val_score(gs, X_train, y_train,
...                              scoring='accuracy', cv=5)
>>> print(f'CV accuracy: {np.mean(scores):.3f} '
...       f'+/- {np.std(scores):.3f}')
CV accuracy: 0.974 +/- 0.015
```

sklearn.metrics

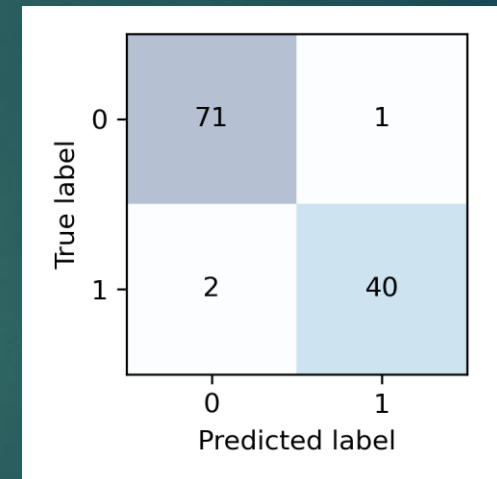
- Score functions, performance metrics, pairwise metrics and distance computations
 - ▶ Example: `confusion_matrix()` function
 - Compute confusion matrix to evaluate the accuracy of a classification

```
>>> from sklearn.metrics import confusion_matrix
>>> pipe_svc.fit(X_train, y_train)
>>> y_pred = pipe_svc.predict(X_test)
➔ >>> confmat = confusion_matrix(y_true=y_test, y_pred=y_pred)
>>> print(confmat)
0 [[ 71  1]
1 [ 2 40]]
```


sklearn.metrics

■ Matplotlib's `matshow()` function

```
>>> fig, ax = plt.subplots(figsize=(2.5, 2.5))
→ >>> ax.matshow(confmat, cmap=plt.cm.Blues, alpha=0.3)
>>> for i in range(confmat.shape[0]):
...     for j in range(confmat.shape[1]):
...         ax.text(x=j, y=i, s=confmat[i, j],
...                 va='center', ha='center')
>>> ax.xaxis.set_ticks_position('bottom')
>>> plt.xlabel('Predicted label')
>>> plt.ylabel('True label')
>>> plt.show()
```



sklearn.metrics

- Example: `precision_score()`, `recall_score()`, `f1_score()`, `matthews_corrcoef()` function

```
>>> from sklearn.metrics import precision_score
>>> from sklearn.metrics import recall_score, f1_score
>>> from sklearn.metrics import matthews_corrcoef

>>> pre_val = precision_score(y_true=y_test, y_pred=y_pred)
>>> print(f'Precision: {pre_val:.3f}')
Precision: 0.976

>>> rec_val = recall_score(y_true=y_test, y_pred=y_pred)
>>> print(f'Recall: {rec_val:.3f}')
Recall: 0.952

>>> f1_val = f1_score(y_true=y_test, y_pred=y_pred)
>>> print(f'F1: {f1_val:.3f}')
F1: 0.964

>>> mcc_val = matthews_corrcoef(y_true=y_test, y_pred=y_pred)
>>> print(f'MCC: {mcc_val:.3f}')
MCC: 0.943
```

sklearn.metrics

► Example: `roc_curve()`, `auc()`, `roc_auc_score()` function

- `roc_curve`: Compute ROC
- `auc`: Compute AUC using the trapezoidal rule
- `roc_auc_score`: Compute ROC AUC from prediction scores

```
>>> from sklearn.datasets import load_breast_cancer
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.metrics import roc_auc_score
>>> X, y = load_breast_cancer(return_X_y=True)
>>> clf = LogisticRegression(solver="newton-cholesky", random_state=0).fit(X, y)
➔ >>> roc_auc_score(y, clf.predict_proba(X)[: , 1])
0.99
```

Probability estimates Class 1

`roc_auc_score` requires the **input scores** for binary classification to represent the probability of the class with the **greater** label (i.e., class 1)

sklearn.metrics

- ▶ Example: `classification_report()` function
 - Build a text report showing the main classification metrics

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 2]
>>> y_pred = [0, 0, 2, 2, 1]
>>> target_names = ['class 0', 'class 1', 'class 2']
➔ >>> print(classification_report(y_true, y_pred, target_names=target_names))
```

	<i>precision</i>	<i>recall</i>	<i>f1-score</i>	<i>support</i>
<i>class 0</i>	0.50	1.00	0.67	1
<i>class 1</i>	0.00	0.00	0.00	1
<i>class 2</i>	1.00	0.67	0.80	3
<i>accuracy</i>			0.60	5
<i>macro avg</i>	0.50	0.56	0.49	5
<i>weighted avg</i>	0.70	0.60	0.61	5

Micro avg corresponds to accuracy and would be the same for all metrics (total FP = total FN).

sklearn.metrics

- ▶ Example: `make_scorer()` function
 - Make a scorer from a performance metric or loss function

```
>>> from sklearn.metrics import make_scorer
>>> c_gamma_range = [0.01, 0.1, 1.0, 10.0]
>>> param_grid = [{'svc__C': c_gamma_range,
...               'svc__kernel': ['linear']},
...               {'svc__C': c_gamma_range,
...               'svc__gamma': c_gamma_range,
...               'svc__kernel': ['rbf']}]
```

➔

```
>>> scorer = make_scorer(f1_score, pos_label=0)
>>> gs = GridSearchCV(estimator=pipe_svc,
...                   param_grid=param_grid,
...                   scoring=scorer,
...                   cv=10)
>>> gs = gs.fit(X_train, y_train)
>>> print(gs.best_score_)
0.986202145696
>>> print(gs.best_params_)
{'svc__C': 10.0, 'svc__gamma': 0.01, 'svc__kernel': 'rbf'}
```

Specify a different positive label

The positive class in scikit-learn is the class labeled as class 1



Tuning and Evaluating a Classifier with Scikit-learn

Mission

- Again the Breast Cancer Wisconsin dataset
- Model Selection
 - ▶ Cross-validation based grid search
- Model Evaluation
 - ▶ Evaluation of the best model
 - Confusion Matrix
 - Classification Report
- Save the Model (and Load the saved Model)
- Sample code: 06_1.py

Model Selection

```
1  import pandas as pd
2  from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
3  from sklearn.impute import SimpleImputer
4  from sklearn.preprocessing import LabelEncoder, StandardScaler
5  from sklearn.feature_selection import SelectFromModel
6  from sklearn.linear_model import LogisticRegression
7  from sklearn.pipeline import Pipeline
8  from sklearn.metrics import classification_report, confusion_matrix
```

```
31  # 4. 建立特徵處理的管線 (Pipeline 只對接特徵矩陣 X)
32  pipeline = Pipeline([
33      ('imputer', SimpleImputer(strategy='mean')),
34      ('scaler', StandardScaler()),
35      ('feature_selection', SelectFromModel(
36          LogisticRegression(l1_ratio=1.0, solver='liblinear', random_state=42)
37      )),
38      ('classifier', LogisticRegression(l1_ratio=0.0, max_iter=2000, random_state=42))
39  ])
```


Model Selection

```
41 # 5. 定義網格 (Search Space)
42 # 注意：在 Pipeline 中必須使用「步驟名稱 + 兩個下底線 + 參數名」
43 param_range = [0.01, 0.1, 1.0, 10.0, 100.0]
44 param_grid = {
45     # 測試特徵選擇 L1 Regularization 強度
46     'feature_selection__estimator__C': param_range,
47
48     # 測試分類器 L2 Regularization 強度
49     'classifier__C': param_range
50 }
51
52 # 6. Model Selection
53 # 將 scoring 設定為 'recall'
54 cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
55 grid_search = GridSearchCV(
56     estimator=pipeline,
57     param_grid=param_grid,
58     cv=cv_strategy,
59     scoring='recall',
60     refit=True,
61     n_jobs=-1
62 )
```

```
64 print("正在進行 5-Fold 交叉驗證與超參數網格搜尋...")
65 grid_search.fit(X_train, y_train)
66
67 print("\n=== Model Selection 最佳化結果 ===")
68 print(f"最佳超參數組合: {grid_search.best_params_}")
69 print(f"最佳 Recall 分數: {grid_search.best_score_:.2%}\n")
```

用訓練集進行
訓練與交叉驗證

5-fold 平均 →

```
=== Model Selection 最佳化結果 ===
最佳超參數組合: {'classifier__C': 10.0, 'feature_selection__estimator__C': 10.0}
最佳 Recall 分數: 95.85%
```

Model Evaluation

$$REC = TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

```
71 # 7. Model Evaluation
72 best_model = grid_search.best_estimator_
73 y_pred = best_model.predict(X_test) ← 用測試集進行評估
74
75 print("=== Model Evaluation 最終驗收報告 ===")
76 print("混淆矩陣 (Confusion Matrix):")
77 print(confusion_matrix(y_test, y_pred))
78
79 # 把 0, 1 還原回原來的字串 B, M, 印出測試報告
80 print("\n詳細分類報告 (Classification Report):")
81 print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))
```

針對個別錯誤 (FN) 進行探討

```
fn_indices = np.where((y_test == 1) & (y_pred == 0))[0]
```

```
=== Model Evaluation 最終驗收報告 ===
混淆矩陣 (Confusion Matrix):
[[68  3]
 [ 1 42]]
詳細分類報告 (Classification Report):
```

	precision	recall	f1-score	support
B	0.99	0.96	0.97	71
M	0.93	0.98	0.95	43
accuracy			0.96	114
macro avg	0.96	0.97	0.96	114
weighted avg	0.97	0.96	0.97	114

Save & Load the Model

```
84 # 8. Save Model
85 # 使用 joblib 進行序列化儲存
86 import joblib
87 # 設定儲存的檔案名稱
88 model_filename = "wdbc_pipeline_lr.pkl"
89 joblib.dump(best_model, model_filename, compress=3)
90 print(f"成功將管線（含預處理與模型）儲存至: '{model_filename}'")
91
92 # 9. Load Model
93 loaded_pipeline = joblib.load(model_filename)
94 print("模型與預處理管線載入成功！")
95 y_pred = loaded_pipeline.predict(X_test)
96 print(confusion_matrix(y_test, y_pred))
```

```
成功將管線（含預處理與模型）儲存至: 'wdbc_pipeline_lr.pkl'
模型與預處理管線載入成功！
[[68  3]
 [ 1 42]]
```



Classwork

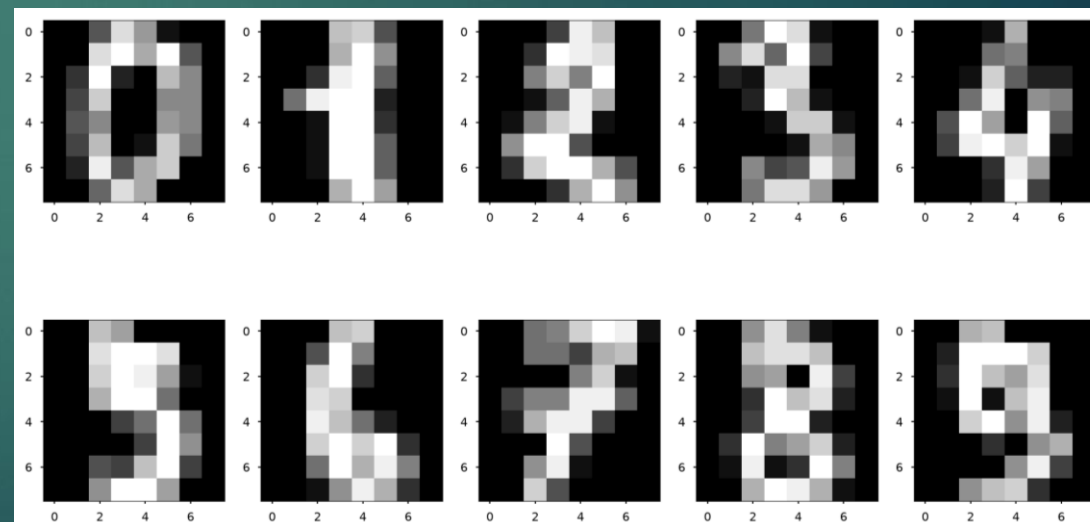
Assignment #3

- 目的：低解析度手寫數字 (0 - 9) 多類別分類器
- 應用情境：
 - ▶ 光學字元辨識 (Optical Character Recognition, OCR)
 - ▶ 假設字元定位與切割已經過預先處理
 - ▶ 本作業僅負責字元辨識任務 (且僅針對 0 - 9 之數字字元)

Assignment #3

- 資料集：[`sklearn.datasets.load\_digits\(\)`](#)
 - ▶ 1797 張 8×8 灰階 (grayscale) 影像
 - ▶ 64 維 (像素值) 特徵 (整數值 [0,16])
 - ▶ 多類別標籤 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) (共 10 類)

```
from sklearn.datasets import load_digits
# 1. 載入資料集
digits = load_digits()
X = digits.data
y = digits.target
```



Assignment #3

■ 實作任務與要求：

- ▶ 以 Logistic Regression 作為分類器演算法
- ▶ 運用資料切分、特徵縮放、特徵選取、模型選取與模型評估
 - 切分比例：80% 訓練 (用於交叉驗證)、20% 測試
 - 至少淘汰 20 個以上的特徵 (即僅使用 44 個以下的像素值)
 - 在測試集 (約 360 筆樣本) 上的整體正確率 (Accuracy) 達 95% 以上
- ▶ 使用 `joblib.dump()` 將訓練好的最佳模型 (指管線) 儲存成檔案 (see p. 53)

```
import joblib
# 設定儲存的檔案名稱
best_model = grid_search.best_estimator_
model_filename = "digits_pipeline_lr.pkl"
joblib.dump(best_model, model_filename, compress=3)
print(f"成功將管線 (含預處理與模型) 儲存至: '{model_filename}')
```


Assignment #3

■ 上傳繳交作業要求：

▶ 將模型評估部分的程式碼 (及模型儲存檔案) 獨立出來進行上傳繳交作業 (打包成單一 .zip 壓縮檔)

- 不要包含任何訓練或模型選取程式碼
- 助教將透過執行模型評估程式批閱作業 (沒有時間進行交叉驗證與微調)

```
1 import joblib
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import classification_report, confusion_matrix
4 import matplotlib.pyplot as plt
5 plt.style.use('seaborn-v0_8-poster')
6
7 from sklearn.datasets import load_digits
8 # 1. 載入資料集
9 digits = load_digits()
10 X = digits.data
11 y = digits.target
```

僅載入 `train_test_split()` 與 `metrics` 相關函式



```
13 # 2. 切分訓練集與測試集 (80% 訓練, 20% 測試)
14 _, X_test, _, y_test = train_test_split(
15     X, y, test_size=0.2, random_state=42
16 )
17
18 # 3. 載入模型
19 model_filename = "digits_pipeline_lr.pkl"
20 best_model = joblib.load(model_filename)
21 print("模型與預處理管線載入成功!")
22
23 # 4. Model Evaluation
24 y_pred = best_model.predict(X_test)
```

僅使用測試集資料

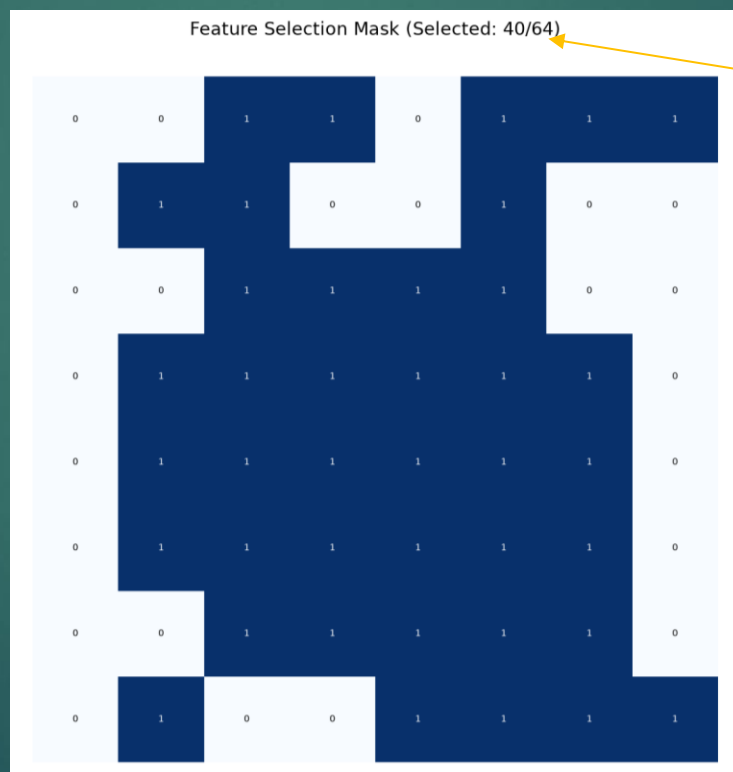
直接載入模型進行評估

Assignment #3

■ 預期結果 1

▶ 視覺化 (特徵選取遮罩) 呈現特徵選取結果

- 0: 淘汰
- 1: 選取
- 標題含選取比例

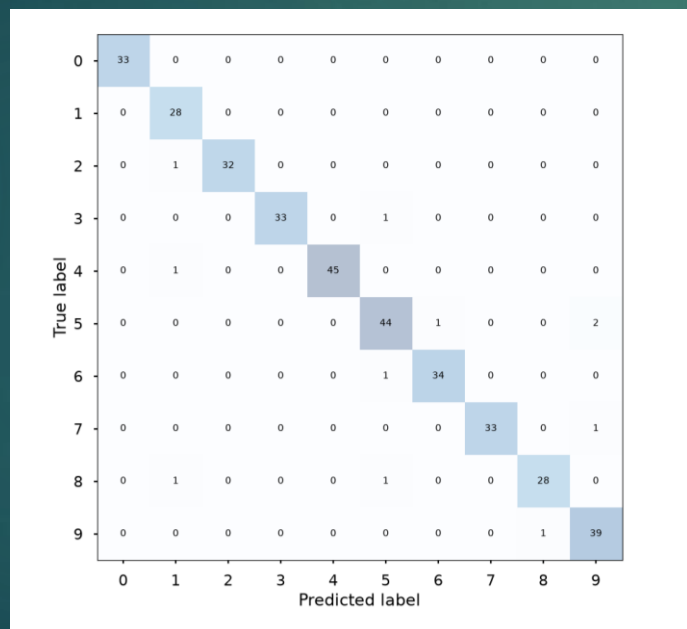


必須在 44/64 以下

Assignment #3

■ 預期結果 2

- ▶ 輸出測試集的分類報告
- ▶ 視覺化呈現混淆矩陣



詳細分類報告 (Classification Report):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.90	1.00	0.95	28
2	1.00	0.97	0.98	33
3	1.00	0.97	0.99	34
4	1.00	0.98	0.99	46
5	0.94	0.94	0.94	47
6	0.97	0.97	0.97	35
7	1.00	0.97	0.99	34
8	0.97	0.93	0.95	30
9	0.93	0.97	0.95	40
accuracy			0.97	360
macro avg	0.97	0.97	0.97	360
weighted avg	0.97	0.97	0.97	360

必須達 0.95 以上

Assignment #3

■ 預期結果 3

▶ 輸出誤判的樣本總數

▶ 繪出所有誤判的樣本

- 標示真實標籤
- 標示預測結果
- 必須與混淆矩陣對應

測試集總共誤判了 11 張圖片。





Thank you